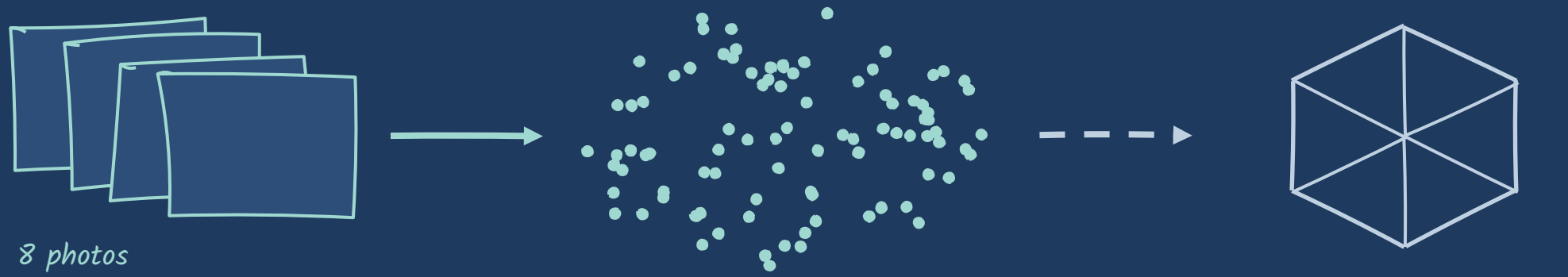
From photos to dense, accurate point clouds

Progress on the first half of the pipeline



8 photos

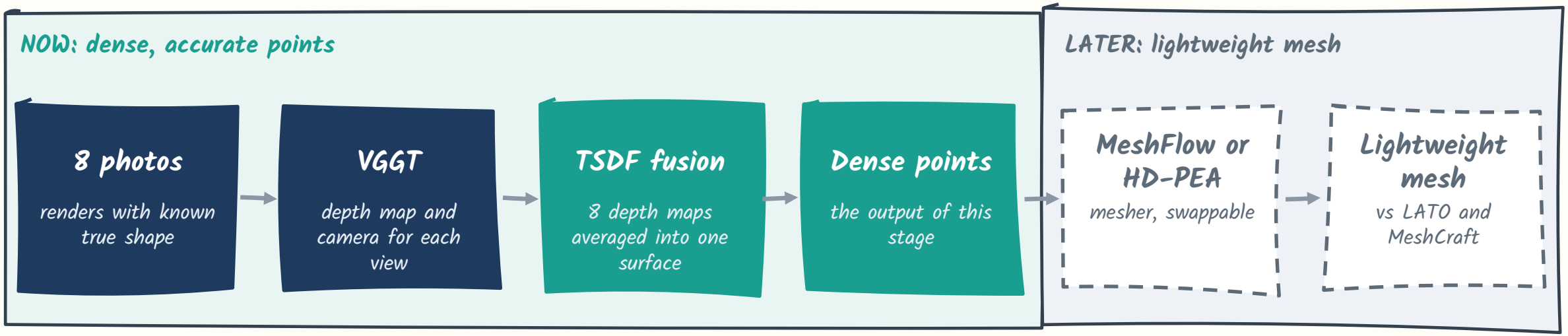
dense, accurate points (now)

lightweight mesh (later)

Fahad Qaseem Khawar · Prof. Zhong's lab · meeting on Monday, Nov 16, 2026

THE PIPELINE

Two components; I am on the first half now



Every part can be swapped: a better point source or a different mesher plugs in without rebuilding the rest.

Agreed on Nov 9: dense, accurate, high-fidelity points first. Lightweight output comes later from the mesher.

SETUP

How I test it

Known true shapes

- 19 ThingiI0K shapes rendered from 8 views (two rings, 5I8 px, textured)
- 7 development shapes and I2 held-out test shapes, never used for tuning

MeshFlow's scoring

- Chamfer and Hausdorff, ×I00, 5,000 samples, shapes scaled to [−0.5, 0.5]
- Plus FI at 0.0I and 0.02, and normal consistency, as you asked

Runs on a laptop

- MacBook M4 Pro, 24 GB
- Per object: VGGT about 25 s, TSDF about 20 s, MeshFlow about 50 s

Three ways to turn VGGT's output into points

VGGT output

8 depth maps and cameras

1

Filter each view

Keep a point only if 2 other views agree

Threw away half the good surface

before

2

TSDF fusion

Average all 8 depth maps in a 3D grid

One clean, closed surface

new default

3

Gaussian centres

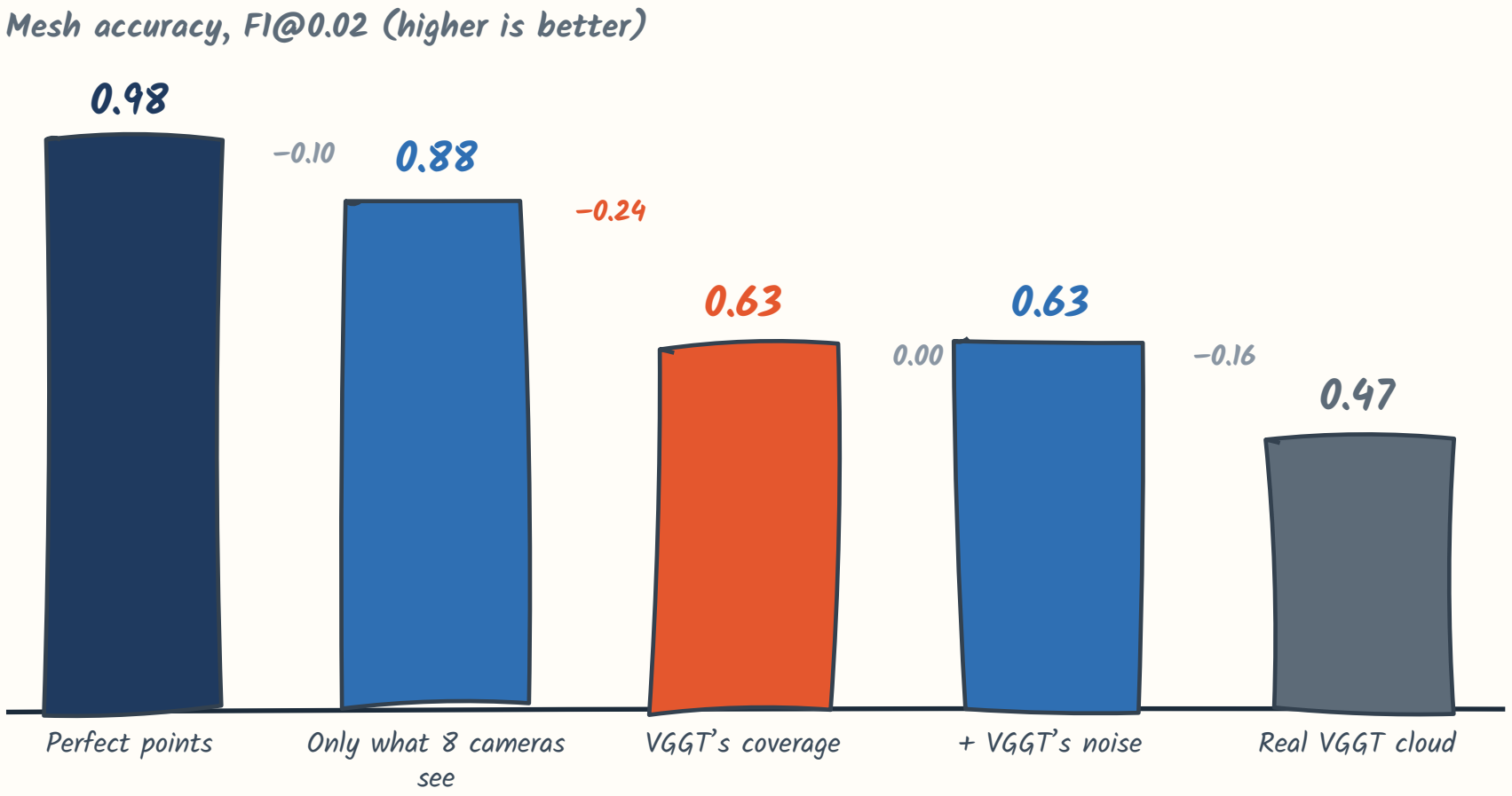
Fit 3D blobs to the photos, keep their centres

Accurate but rough, 3× vertices

tested

Each source goes into the same MeshFlow, so only the points change

The biggest loss is surface VGGT never recovers



48%

of the error gap: surface VGGT never recovers

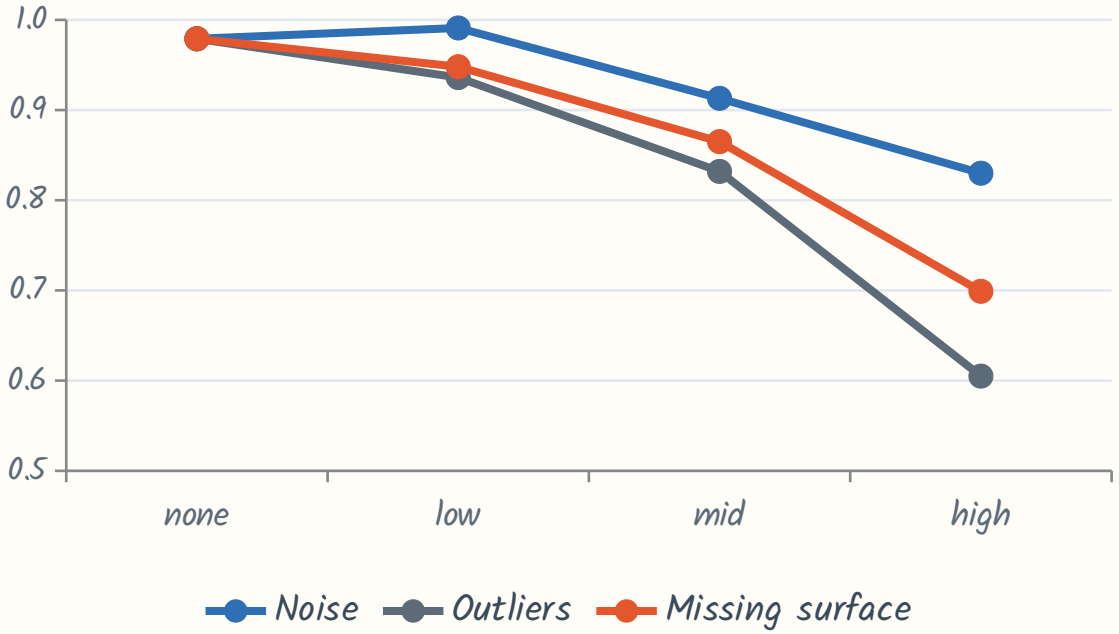
4%

comes from VGGT's noise, almost nothing

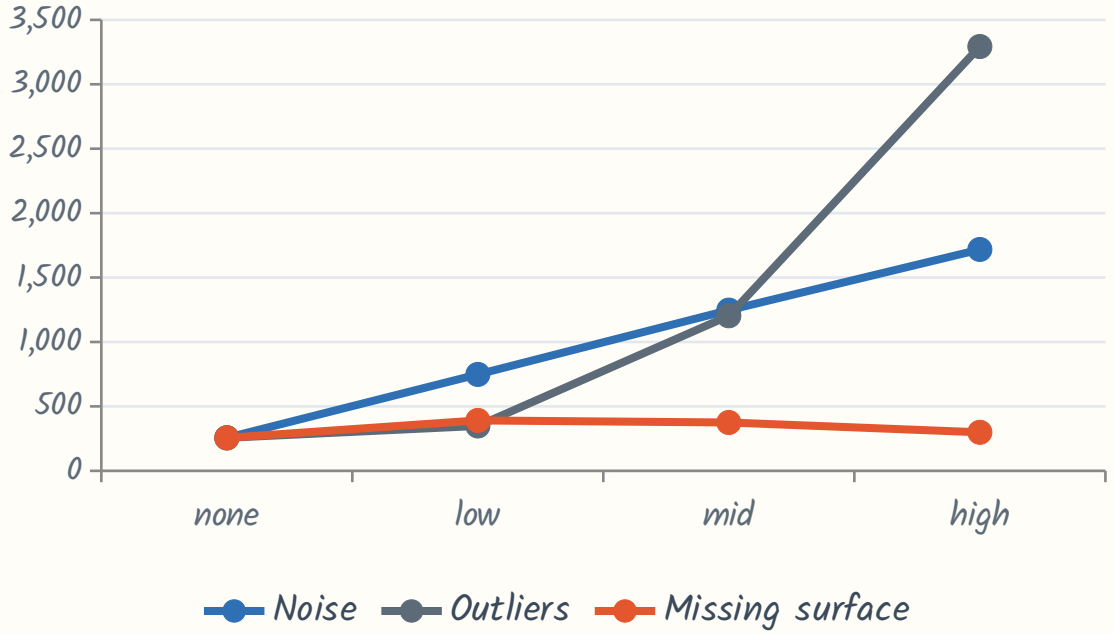
So the fix had to recover more surface, not remove more noise

MeshFlow tolerates noise, not missing surface

Accuracy, F1@0.02 (higher is better)



Vertices in the mesh

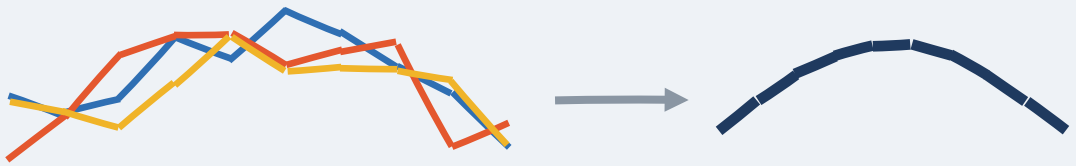


low / mid / high = noise 0.5 / 1 / 2% of size · outliers 1 / 5 / 10% · missing 10 / 25 / 50%. MeshFlow on damaged perfect points, 7 shapes.

Noise mostly adds vertices (MeshFlow reads bumps as detail); missing surface costs accuracy

THE FIX

TSDF fusion: average the views so their errors cancel



3 of the 8 depth guesses, each a little off

one averaged surface

Each block in a 3D grid keeps a running vote: in front of the surface or behind it. The surface is where the votes switch. A standard method from 1996, about 20 s per object.

Chamfer error

9.58 → **6.24** −35%

F1@0.02

0.44 → **0.60** +0.16

Vertices

1,440 → **933** −35%

Better on all 12 held-out test shapes; now the default point source

POINTS FIRST: SCORED AGAINST THE TRUE SHAPE

TSDF gives the best point cloud we have

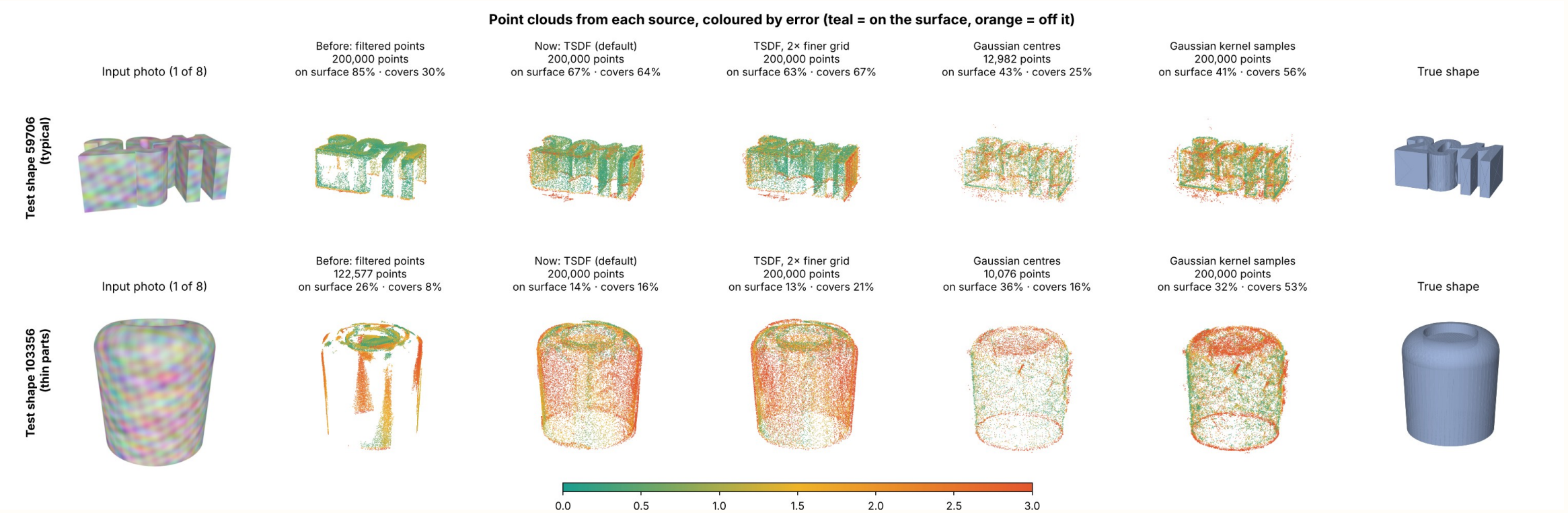
Point source (12 test shapes)	Chamfer ↓	F1@0.01 ↑	F1@0.02 ↑	Normals ↑	Coverage	Points
Filtered VGGT points (before)	8.37	0.38	0.56	0.81	43%	186k
TSDF fusion (default)	5.61	0.47	0.71	0.81	76%	200k
Gaussian centres	5.74	0.26	0.61	0.77	50%	11.5k
Gaussian kernel samples	5.80	0.39	0.68	0.76	80%	200k

- TSDF nearly doubles coverage (43% to 76%) and is the most accurate on Chamfer and F1
- Gaussian kernels reach the most surface (80%) but are less precise, with rougher normals
- Coverage = share of the true surface with a point within 1% of the object size

From now on the point cloud is the main result; MeshFlow meshes are a secondary check

POINT CLOUDS, COLOURED BY ERROR

What the point clouds look like

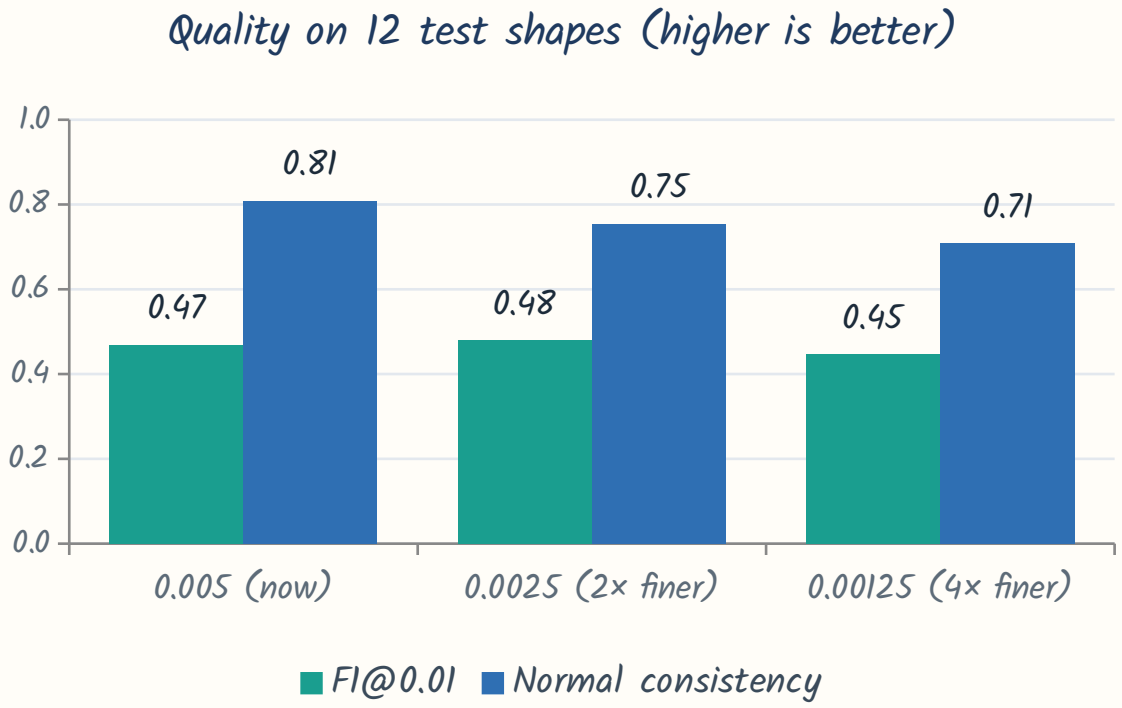


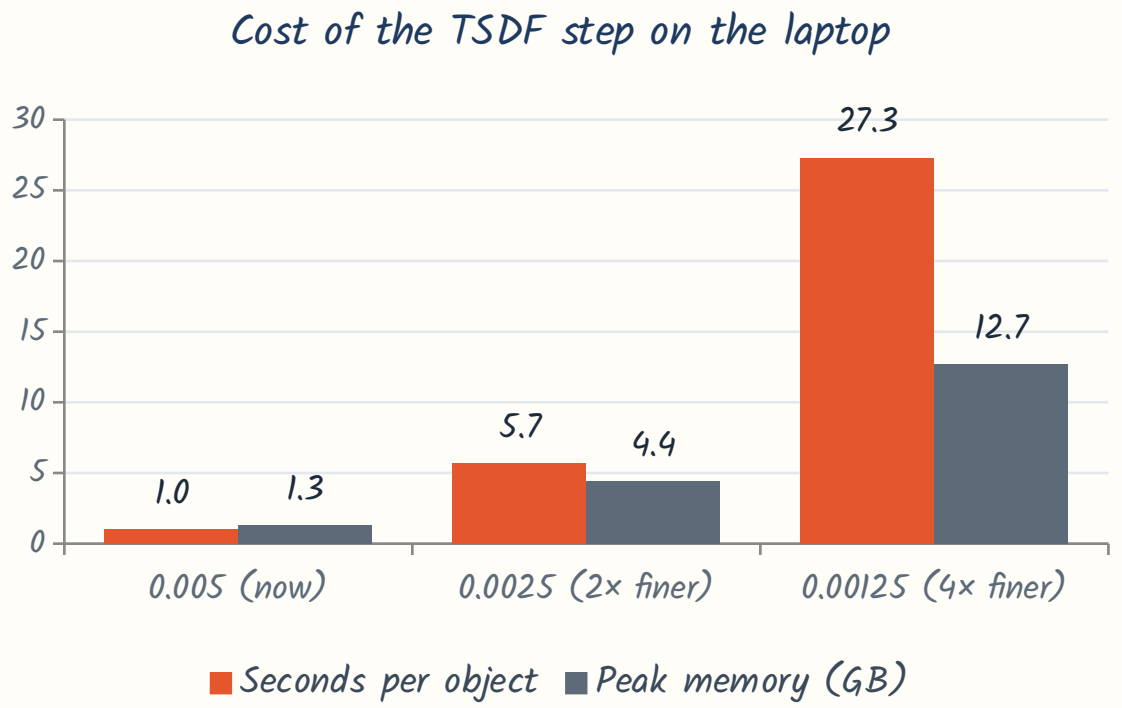
Teal = on the true surface, orange = 3% of the object's size away. "On surface" and "covers" use a 1% distance, computed for this picture only.

TSDF fills the object; on thin walls it drifts off, and Gaussian kernel samples stay closer

DENSITY SWEEP

Finer TSDF voxels do not help with 8 views

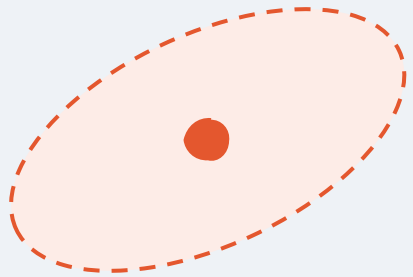




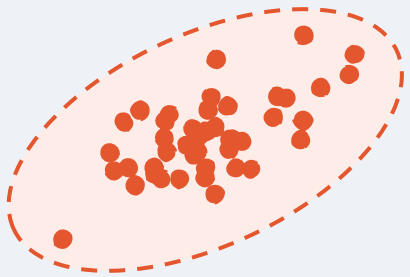
IM points instead of 200k: +0.3 s and the same accuracy; only a denser cloud (all-points recall 0.47 to 0.49).

The limit is VGGT's depth maps, not the fusion grid; the default stays at voxel 0.005

Sampling across each Gaussian beats its centre



Centre only: one point per Gaussian



Kernel sampling: points spread by its shape and opacity

Your Nov 9 suggestion: use the whole Gaussian, not just its centre.

12 of 12 test shapes better than centres (F1@0.01 0.26 to 0.39)

80% coverage, the most surface of any source

Trails TSDF on average, but wins on thin parts:

Shape (F1@0.01)	TSDF	Kernels
thingi_103356	0.14	**0.37**
thingi_77949	0.18	**0.28**
thingi_593381	0.24	**0.33**

Next: TSDF everywhere, plus Gaussian kernels where TSDF loses thin parts

SECONDARY CHECK: AFTER MESHFLOW

After MeshFlow, TSDF still wins

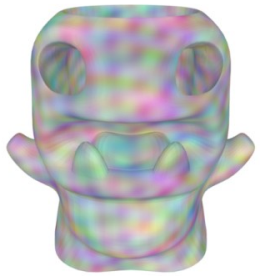
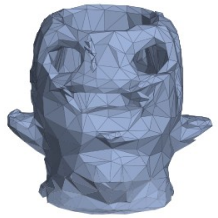
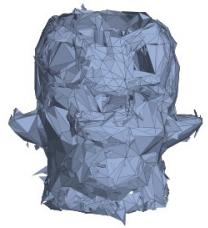
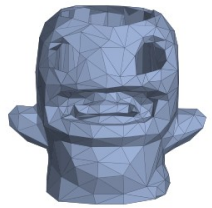
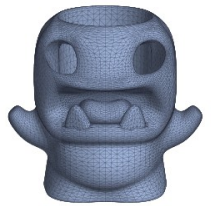
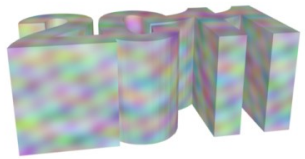
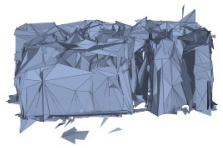
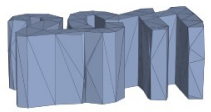
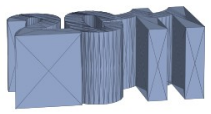
Point source (12 test shapes)	Chamfer ↓	F1@0.02 ↑	Normals ↑	Vertices
Filtered VGGT points (before)	9.58	0.44	0.81	1,440
TSDF fusion (new default)	6.24	0.60	0.82	933
Gaussian centres	6.44	0.58	0.74	2,920
Perfect points (ceiling)	2.93	0.98	0.94	265

- TSDF: a third less error and a third fewer vertices than before
- Gaussian centres: about as accurate, but rougher normals and 3× the vertices
- Caveat: the Gaussians were started from the TSDF points, so not a fully independent test

OUTPUTS

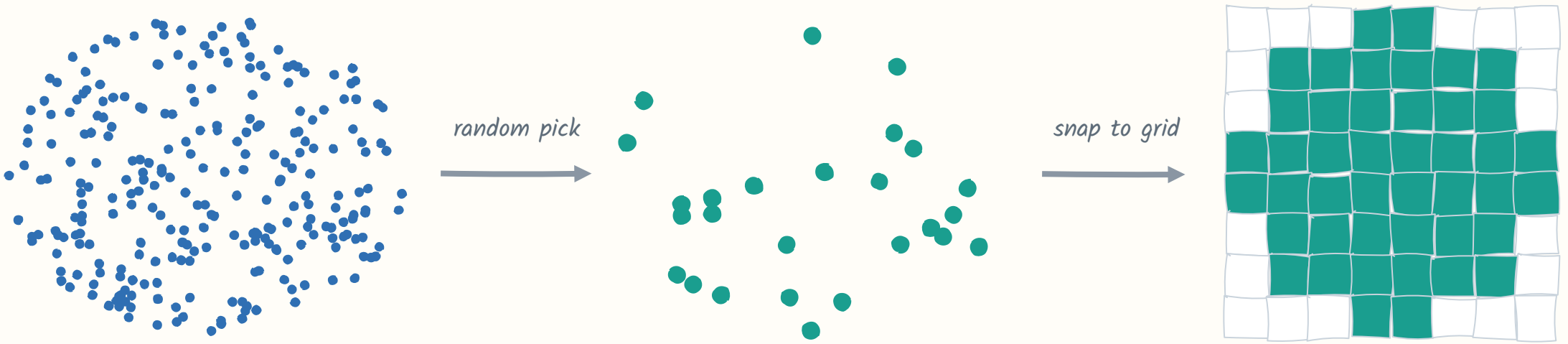
What the meshes look like

MeshFlow output from different inputs (same seed)

	Input photo (1 of 8)	Before: filtered points 918 vertices	Now: TSDF points 982 vertices	Gaussian centres 2,697 vertices	Ceiling: perfect points 361 vertices	True shape 8,988 vertices
Pencil pot (tuning shape)						
	Input photo (1 of 8)	Before: filtered points 806 vertices	Now: TSDF points 753 vertices	Gaussian centres 1,892 vertices	Ceiling: perfect points 158 vertices	True shape 536 vertices
Test shape 59706 (never tuned on)						

Every mesh is made by MeshFlow from a point cloud; the vertex drop is generation, not QEM

MeshFlow reads only 4,096 points on a 32^3 grid



Our cloud: up to 200,000 points

4,096 points

32 × 32 × 32 grid

Can it read more? Not without retraining: the network has exactly 4,096 input slots, and the 32^3 grid is built into its trained weights.

For the mesher, accuracy and coverage matter more than density; a dense cloud still matters as this stage's own output and for HD-PEA

Not without retraining, so make the 4,096 count



Pick an evenly spread 4,096

tested

Hand MeshFlow exactly 4,096 points spread out (farthest-point sampling) instead of a random pick, so thin parts are not skipped



Make those points accurate

in progress

TSDF, plus Gaussian kernels on thin parts: cleaner points matter more than more points



Dense cloud into HD-PEA

later

HD-PEA takes points and normals directly, so the full dense cloud pays off there



Retrain MeshFlow for more points

future idea

A larger input on the lab GPU would be a real mesh-side contribution

The network has exactly 4,096 input slots and the 32^3 grid is built into its trained weights

TESTED: AN EVENLY SPREAD 4,096

Even spreading gives the same accuracy, 4x the vertices

Input (12 test shapes)	Chamfer ↓	F1@0.02 ↑	Normals ↑	Vertices
TSDF, random 4,096 (now)	6.24	0.60	0.82	933
TSDF, evenly spread 4,096	6.31	0.60	0.83	3,862
TSDF + kernels, random 4,096	7.17	0.52	0.72	1,837
TSDF + kernels, spread 4,096	6.49	0.57	0.65	3,707

4x

the vertices for the same accuracy: MeshFlow keeps almost one vertex per input point when they are evenly spread

- Accuracy is unchanged (within the seed-to-seed spread); MeshFlow likely expects random surface samples, as in its own code
- The TSDF + kernel union is worse either way: about half its spread points sit off the surface, so it needs cleaning first

Keep MeshFlow's own random pick; the next lever is cleaner points, not how they are picked

FOLLOW-UP

Answers to your questions from Nov 9

1

How is the mesh downsized? QEM?

No simplification step. MeshFlow generates a new mesh, guided by the points, in the artist style it learned.

2

Marching cubes for mesh extraction?

Every mesh comes from MeshFlow. Marching cubes runs only inside TSDF, to build the surface I sample points from.

3

Why are the Gaussian-centre meshes bumpy?

From 8 views, splatting fits appearance, so many centres sit slightly off the surface; MeshFlow reads that as detail.

4

Why the ceiling column?

MeshFlow on perfect points: it shows the mesher is fine, so the gap is in the points.

Next steps on the points side



Clean TSDF plus Gaussian kernels

Kernel samples only where the TSDF surface is missing, with stray samples removed

next



16 views on the lab GPU

Better depth maps are now the main lever; the laptop fits 8 views

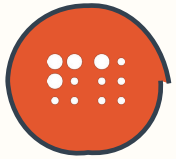
needs GPU



ShapeNet and Toys4K

Render scripts are ready; both datasets need an access request

requesting access



Dense output

IM points cost nothing extra if you want a denser cloud

your call

Five decisions I need from you

1. **Dataset:** ShapeNet, or Toys4K (MeshFlow's own test set)?
2. **Final baselines:** LATO (March 2026) or LATO.2 (July 2026), plus MeshCraft?
3. **Dense output:** Switch the default to IM points? Cheap, but denser rather than more accurate.
4. **Camera focal length:** Is using the known focal length acceptable? Phones record it.
5. **Lab GPU:** Access for 16-view runs; the laptop fits 8 views.